

**Algoritmi e Strutture Dati – Parte A – 13/06/25**

**Esercizio -1** Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

**Esercizio 0** Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

**A1 – Complessità – Punti  $\geq 8$** 

Trovare i limiti asintotici superiore e inferiore più stretti possibili per la seguente famiglia di equazioni di ricorrenza (assumendo che  $0 < \delta < 1$ ):.

$$T(n) = \begin{cases} T(\lfloor \delta n \rfloor) + n^{\delta+1} & n > 1 \\ 1 & n \leq 1 \end{cases}$$

**A2 – 1D Board – Punti  $\geq 8$** 

Sia data una scacchiera monodimensionale con celle numerate da 1 a  $n$ , rappresentata da un vettore booleano  $A$  dove le celle con valore **true** sono percorribili e quelle con valore **false** non lo sono. Una pedina parte dalla cella  $start$  ( $1 \leq start \leq n$  e  $A[start] = \text{true}$ ) e può muoversi come segue:

- **forward**: se si trova nella cella  $x$  e  $x + f \leq n$ , può spostarsi nella cella  $x + f$ ;
- **backward**: se si trova nella cella  $x$  e  $x - b \geq 1$ , può spostarsi nella cella  $x - b$ ;
- la mossa è consentita solo se la cella di arrivo è percorribile, cioè  $A[\text{cella di arrivo}] = \text{true}$ .

Scrivere un algoritmo

**int** minSteps(**boolean**[]  $A$ , **int**  $n$ , **int**  $start$ , **int**  $f$ , **int**  $b$ )

che restituisca il numero minimo di mosse necessarie per raggiungere la cella 1, o  $-1$  se non è raggiungibile.

Discutere informalmente la correttezza dell'algoritmo proposto e calcolarne la complessità computazionale.

**A3 – Stringhe pareggiate – Punti  $\geq 8$** 

Una stringa si dice *pareggiata* se per ogni carattere minuscolo che compare in essa è presente anche il corrispondente carattere maiuscolo e viceversa. Per esempio, "aCcA" e "caCA" sono pareggiate, "ABa" non lo è perché contiene il carattere "B" ma non contiene il carattere "b". Per comodità, assumete che esista una funzione `swapcase(item c)` che prende in input un carattere  $c$  e

restituisce il carattere corrispondente con l'altro tipo di capitalizzazione (maiuscolo se è minuscolo, e viceversa). Scrivere un algoritmo

**int** largestPaired(**item**[]  $S$ , **int**  $n$ )

che restituisce la lunghezza della più lunga sottostringa pareggiata contenuta in  $S$  (non sottosequenza: la sottostringa deve essere contigua).

Discutere informalmente la correttezza dell'algoritmo proposto e calcolare la sua complessità computazionale.

Per esempio:

- la più lunga sottostringa pareggiata contenuta in "BaCcAno" è "aCcA" e l'algoritmo deve restituire 4;
- nel caso della stringa "aAa", la più lunga sottostringa pareggiata è la stringa stessa, quindi l'algoritmo deve restituire 3;
- la più lunga sottostringa pareggiata in "torTA" non è "tT" (una sottosequenza), ma la stringa vuota e quindi l'algoritmo deve restituire 0.

## Algoritmi e Strutture Dati – Parte B – 13/06/25

**Esercizio -1** Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

**Esercizio 0** Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

### B1 – Braccia rubate alla ricerca – Punti $\geq 8$

Il DISI ha deciso di organizzare le pulizie degli uffici senza ricorrere a imprese esterne, ma distribuendo equamente il lavoro tra dottorandi e assegnisti.

Le informazioni a disposizione sono:

- l'insieme  $O$  degli uffici da pulire;
- l'insieme  $L$  delle persone disponibili (dottorandi e assegnisti);
- ogni persona può pulire al massimo  $m$  uffici;
- ogni persona ha indicato l'elenco  $A_\ell \subseteq O$  degli uffici che è disposta a pulire.

L'obiettivo è verificare se è possibile assegnare a ogni ufficio una persona disponibile in modo che:

- ogni ufficio venga pulito esattamente una volta;
- ogni persona pulisca al più  $m$  uffici;
- ogni assegnazione rispetti la disponibilità dichiarata.

Descrivere un algoritmo che, dati gli insiemi e i vincoli sopra, determini se esiste una pianificazione delle pulizie valida. Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

### B2 – Dollar word – Punti $\geq 10$

Supponiamo di associare il valore 1 centesimo alla lettera "a", 2 centesimi alla lettera "b", ..., 26 centesimi alla lettera "z", considerando solo lettere minuscole dell'alfabeto inglese. Nel suo seminario tenuto a Trento nel Maggio 2025, Gary Stager ha accennato alle "dollar words", ovvero parole la cui somma dei valori delle lettere è pari a un dollaro, cioè 100 centesimi. Per esempio, "whiskey" è una dollar word.

Scrivere una funzione

```
int countWords(int T)
```

che prende in input un valore  $T$  e restituisce il numero di stringhe che valgono  $T$  centesimi, di arbitraria lunghezza.

Discutere informalmente la correttezza dell'algoritmo proposto e calcolare la sua complessità computazionale.

Per esempio:

- le stringhe che valgono 1¢ sono 1: "a"
- le stringhe che valgono 2¢ sono 2: "aa", "b"
- le stringhe che valgono 3¢ sono 4: "aaa", "ab", "ba", "c"
- le stringhe che valgono 4¢ sono 8: "aaaa", "aab", "aba", "baa", "bb", "ca", "ac", "d"

### B3 – Tuple ben ordinate – Punti $\geq 12$

Sia  $A$  un vettore contenente  $n$  interi. Una *tupla ben ordinata di dimensione  $k$*  (con  $1 \leq k \leq n$ ) è una sequenza di valori

$$(A[i_1], A[i_2], \dots, A[i_k])$$

tale che gli indici soddisfano  $i_1 < i_2 < \dots < i_k$  e i corrispondenti valori sono ordinati strettamente in modo crescente o decrescente, ovvero:

$$A[i_1] < A[i_2] < \dots < A[i_k]$$

oppure

$$A[i_1] > A[i_2] > \dots > A[i_k].$$

Scrivere un algoritmo

```
printTuples(int A[], int n, int k)
```

che stampi *tutte* le tuple ben ordinate di dimensione  $t$ , per ogni  $t$  tale che  $2 \leq t \leq k$ , presenti nel vettore  $A$ .

Discutere informalmente la correttezza dell'algoritmo proposto e calcolare la sua complessità computazionale.

Per esempio, se  $A = [2, 5, 3, 4, 1]$  e  $k = 3$ , l'algoritmo deve produrre le tuple  $[3, 4]$ ,  $[2, 4]$ ,  $[2, 3]$ ,  $[2, 3, 4]$ ,  $[2, 5]$ ,  $[4, 1]$ ,  $[3, 1]$ ,  $[5, 1]$ ,  $[5, 4]$ ,  $[5, 4, 1]$ ,  $[5, 3]$ ,  $[5, 3, 1]$ ,  $[2, 1]$ .